

Core Logic & Control Flow

Conditional Statements

Mastering decision-making in JavaScript: `if/else`, `switch`, and the `ternary operator`.

Session Agenda

1

Conditional Concept

What conditionals are & real-world decision logic.

2

if & if...else Statements

Basic syntax, evaluation flow, and binary branching.

3

if...else if...else Chains

Multi-condition branching & common condition ordering bugs.

4

Nested if Statements

Hierarchical logic and simplification via logical AND (&&).

5

switch Statement

Exact matching, fall-through bug, and intentional fall-through.

6

Ternary Operator & Matrix

One-line expressions and "Which one to use when" comparison.

Quick Recap: Day 2 Foundations



What We Covered Previously

Data Types: Primitive vs Non-primitive values.

Type Conversion & Coercion: Explicit vs implicit logic.

Comparison Operators: `>`, `<`, `>=`, `<=`, `===`.

Logical Operators: `&&` (AND), `||` (OR), `!` (NOT).



Why It Matters Today

Conditionals rely directly on **Boolean outputs** (`true` / `false`).

Truthy & Falsy values determine which branch executes.

Comparison operators form the core condition rules.

Enables real logic like "Show dashboard if logged in."

What is a Conditional Statement?

A conditional statement controls execution flow based on whether a specified condition evaluates to **true** or **false**.

REAL-WORLD ANALOGY

"If it is raining outside, take an umbrella. Otherwise, do not take an umbrella."

CORE PROGRAMMING PATTERN

IF (condition is true) → Execute Block A
ELSE → Execute Block B

3 Primary Conditional Structures in JavaScript:

1. `if / else`

2. `switch`

3. `ternary operator (? :)`

The basic `if` Statement

```
let age = 20;

if (age ≥ 18) {
  console.log("You are eligible to vote.");
}

console.log("Program continues...");
```

Structure Rules

- Condition goes in round brackets ().
- Executable block goes in curly braces {}.

💡 Execution Behavior

If condition is **false** (e.g., `age = 15`), the block is entirely skipped and program continues smoothly.

The `if...else` Statement

```
let temperature = 35;

if (temperature > 30) {
  console.log("It's hot outside, drink water!");
} else {
  console.log("Weather is pleasant today.");
}
```

Binary Handling

Guarantees that exactly **one** block will run: either `if` (when true) or `else` (when false).

Key Takeaway

`else` requires no condition — it automatically catches all cases where `if` condition fails.

The `if...else if...else` Chain

```
let marks = 75;

if (marks ≥ 90) {
  console.log("Grade: A+");
} else if (marks ≥ 80) {
  console.log("Grade: A");
} else if (marks ≥ 70) {
  console.log("Grade: B");
} else {
  console.log("Grade: F");
}
```

Top-Down Evaluation

Evaluates conditions sequentially from top to bottom.

⚡ First Match Wins

As soon as a condition evaluates to **true**, its block runs and all subsequent conditions are skipped entirely.

❌ Common Mistake: Wrong Condition Order

```
let marks = 95; // Student scored 95!  
  
// WRONG ORDER: General condition placed first  
if (marks ≥ 60) {  
  console.log("Grade: C"); // ❌ Triggers first!  
} else if (marks ≥ 90) {  
  console.log("Grade: A+"); // ⚠️ NEVER REACHED!  
}
```

Why This Bug Happens

Since `marks = 95` is `>= 60`, the very first condition evaluates to **true**.

JavaScript executes `Grade: C` and exits the chain — `Grade: A+` becomes dead code.

✓ Fixed Version: Specific-to-General Order

```
let marks = 95;

// FIXED ORDER: Strict / Specific conditions first
if (marks ≥ 90) {
  console.log("Grade: A+"); // ✓ Executes correctly!
} else if (marks ≥ 60) {
  console.log("Grade: C");
}
```

Golden Rule of Chains

Always order conditions from **most specific / highest threshold** down to **most general / lowest threshold**.

Nested if Statements

```
let age = 20;
let hasID = true;

if (age ≥ 18) {
  if (hasID) {
    console.log("Entry allowed.");
  } else {
    console.log("Please show your ID.");
  }
} else {
  console.log("You must be 18+ to enter.");
}
```

Definition

Placing an `if` statement inside another `if` statement to evaluate multi-stage dependent requirements.

⚠ Caution

Avoid nesting deeper than 2–3 levels. Excessive nesting leads to unreadable code ("Pyramid of Doom").

Simplifying Nested if with &&

```
let age = 20;
let hasID = true;

// Flattened logic using Logical AND (&&)
if (age ≥ 18 && hasID) {
  console.log("Entry allowed.");
} else {
  console.log("Entry denied.");
}
```

Why This Is Better

- ✓ **Cleaner syntax:** Reduces indentation.
- ✓ **More readable:** Intent is clear in one line.
- ✓ **Less bug-prone:** Simpler branching paths.

The switch Statement

```
let day = 3;

switch (day) {
  case 1: console.log("Monday"); break;
  case 2: console.log("Tuesday"); break;
  case 3: console.log("Wednesday"); break;
  default:
    console.log("Weekend or invalid day");
}
```

When to Use

Ideal when evaluating a **single variable against multiple discrete exact values**.

Key Components

- **case**: Specific matching value.
- **break**: Stops switch execution.
- **default**: Fallback if no match.

❌ The Infamous "Fall-Through" Bug

```
let day = 2;

switch (day) {
  case 1: console.log("Monday");
  case 2: console.log("Tuesday"); // Matches!
  case 3: console.log("Wednesday"); // ALSO RUNS!
  default: console.log("Invalid day"); // ALSO RUNS!
}

// Output: Tuesday, Wednesday, Invalid day
```

What Happened Here?

Without **break** statements, once a case matches, JavaScript executes **ALL subsequent cases** regardless of whether they match!

✓ Fixed Version: Always Add break

```
let day = 2;

switch (day) {
  case 1: console.log("Monday"); break;
  case 2: console.log("Tuesday"); break; // ✓ Exits cleanly
  case 3: console.log("Wednesday"); break;
  default: console.log("Invalid day");
}

// Output: Tuesday
```

Rule of Thumb

Every single case block should end with a **break** statement unless you are intentionally implementing multi-case grouping.

Intentional Fall-Through

```
let day = 6;

switch (day) {
  case 6:
  case 7:
    console.log("It's the weekend!");
    break;
  default:
    console.log("It's a weekday.");
}
```

Valid Use Case

When multiple values (e.g., Saturday 6 and Sunday 7) should share the exact same output, intentionally omit the break between them.

The Ternary Operator (? :)

`condition ? valueIfTrue : valueIfFalse`

```
let age = 20;

// One-line conditional decision
let result = age ≥ 18 ? "Eligible" : "Not eligible";

console.log(result); // "Eligible"
```

Key Advantages

- Extremely concise shortcut for simple binary choices.
- Evaluates directly to a **value** that can be assigned to a variable or logged inline.

COMPARISON

if...else vs Ternary Operator

Standard if...else (5 lines)

```
let message;  
  
if (age ≥ 18) {  
  message = "Adult";  
} else {  
  message = "Minor";  
}
```

Equivalent Ternary (1 line)

```
// Concise & clean  
let message = age ≥ 18 ? "Adult" : "Minor";
```

Directly inside methods: `console.log(marks ≥ 50 ? "Pass" : "Fail");`

Chained Ternary Operators

```
let marks = 75;

// Chained ternary logic
let grade = marks ≥ 90 ? "A+"
               : marks ≥ 70 ? "B"
               : "C";

console.log(grade); // "B"
```

⚠ Readability Warning

While chaining is possible, it rapidly degrades readability.

Rule: For multi-condition logic, always prefer `if...else if...else` over chained ternaries.

Which One to Use, When?

Scenario / Requirement	Best Choice	Why
Simple true/false binary choice in 1 line	Ternary Operator	Maximum conciseness and inline evaluation
2–3 conditions with distinct outcome logic	if / else if / else	Clear visual structure and standard convention
Many conditions checking ONE variable's exact value	switch statement	Cleaner than 10+ else if blocks
Complex, range-based, or multi-variable logic	if / else if / else	Supports logical operators (&&,) & complex expressions



Practice Task

Task 1: Positive / Negative / Zero

Write an `if...else if...else` statement that checks whether a given number is positive, negative, or zero.

Task 2: Season Finder

Write a `switch` statement taking a month number (1-12) and logging the season (Winter, Spring, Summer, Fall). Include breaks!

Task 3: Driving Eligibility

Write a ternary operator that checks if a person can drive (`age >= 16`) and logs "Can Drive" or "Cannot Drive".

Task 4: Bonus Challenge

Rewrite Task 1 (positive/negative/zero check) using a chained ternary operator.

Session Recap

✓ **if statement:** Single condition branching.

✓ **if...else:** Handling true vs false paths.

✓ **if...else if:** Ordering specific conditions first.

✓ **Nested ifs:** Simplifying with logical &&.

✓ **switch:** Discrete values & preventing fall-through.

✓ **Ternary operator:** One-line expression syntax.

Homework & Next Session



Homework Assignment

1. Complete all 4 practice tasks from today's deck.
2. Build an **ATM Cash Withdrawal Simulation**:
 - Check correct PIN code.
 - Validate requested withdrawal amount against balance.
 - Log clear success or error messages.



Up Next: Day 4

Loops in JavaScript

Automating repetitive tasks using: for, while, do-while, break, and continue.



Great Job Today!

Share your task code in the comments below.
Don't forget to Like, Subscribe, and Practice!

Allah Hafız & See You in Day 4!